AES 128 Implementation

Suparna Das(Crs2323)

Indian Statistical Institute, Kolkata

1 Introduction

1.1 Introduction to AES (Advanced Encryption Standard)

The Advanced Encryption Standard (AES) was established by the National Institute of Standards and Technology (NIST) in 2001 to replace the aging Data Encryption Standard (DES). AES is a symmetric encryption algorithm that supports key sizes of 128, 192, and 256 bits. AES-128, with a 128-bit key length, is widely used due to its robust security and efficient performance.

1.2 Need for Symmetric Encryption

Symmetric encryption, also known as private-key encryption, uses the same key for both encrypting and decrypting data. This method is efficient and typically faster than asymmetric encryption, making it ideal for real-time applications and large data volumes. However, the key management challenge is significant: both parties must securely share and protect the secret key.

2 AES-128 Overview

2.1 AES Basics

AES operates on fixed-size blocks of 128 bits and supports key lengths of 128, 192, and 256 bits. AES-128, specifically, uses a 128-bit key and performs encryption through 10 rounds of processing for data blocks

2.2 AES-128 Specifics

- **Key length:** 128 bits
- **Block size:** 128 bits
- **Number of rounds:** 10 rounds

In AES-128, the encryption process involves several key steps, including SubBytes, ShiftRows, MixColumns, and AddRoundKey operations. The decryption process is similar but involves the inverse of these operations.

3 AES Algorithm Structure

- **Key expansion:** Key expansion is the process of generating round keys from the original cipher key. These round keys are used in each round of the AES encryption and decryption processes.

  Key expansion Algorithm:

  - **Initial Key Copy:**Copy the original key into the first part of the round keys.
  - **Generation of Round Keys:** For each subsequent round, generate a new round key based on the previous round key using functions like RotWord, SubWord, and XOR with Rcon (round constants).

- **Encryption Process:**
  - **Initial round:**
    - **Add round key:** XOR the plaintext with the first round key.
  - **Main rounds**(for each round):
    - **SubBytes:** Substitute bytes in the state matrix using an S-Box
    - **ShiftRows:** Shift rows of the state matrix to the left
    - **MixColumns:** Mix the columns of the state matrix using a linear transformation.
    - **AddRoundKey:** XOR the state matrix with the round key
  - **Final Round:**
    - **SubBytes:** Substitute bytes in the state matrix using an S-Box
    - **ShiftRows:** Shift rows of the state matrix to the left
    - **AddRoundKey:** XOR the state matrix with the round key

- **Decryption process:**
  The decryption process involves reversing the operations of encryption
  - **Initial round:**
    - **Add round key:** XOR the ciphertext with the final round key.
  - **Main rounds**(for each round):
    - **InvShiftRows:** Shift rows to the right.
    - **InvSubBytes:** Substitute bytes using the inverse S-Box.
    - **AddRoundKey:** XOR the state matrix with the round key
    - **InvMixColumns:** Inverse of MixColumns operation.
  - **Final Round:**
    - **InvShiftRows:** Shift rows to the right.
    - **InvSubBytes:** Substitute bytes using the inverse S-Box.
    - **AddRoundKey:** XOR the state matrix with the round key

4 Description of the AES algorithm:

AES is an iterated block cipher with a fixed block size of 128 and a variable key length. The different transformations operate on the intermediate results, called state. The state is a rectangular array of bytes and since the block size is 128 bits, which is 16 bytes, the rectangular array is of dimensions 4x4. (In the Rijndael version with variable block size, the row size is fixed to four and the number of columns vary. The number of columns is the block size divided by 32 and denoted Nb). The cipher key is similarly pictured as a rectangular array with four rows. The number of columns of the cipher key, denoted Nk, is equal to the key length divided by 32.

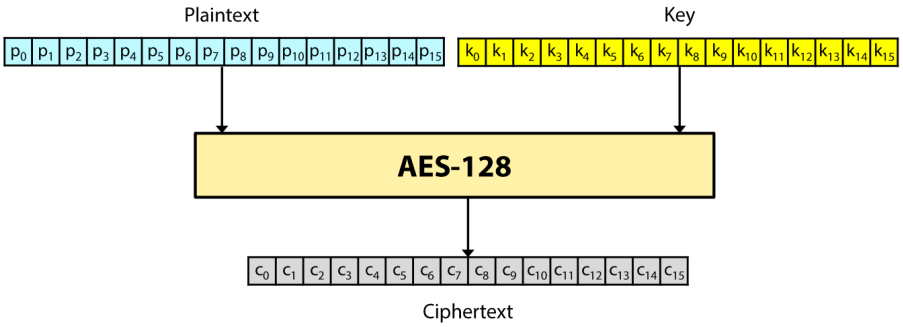


Figure 1: AES-128

A State:

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$

A Key:

$k_{0,0}$	$k_{0,1}$	$k_{0,2}$	$k_{0,3}$
$k_{1,0}$	$k_{1,1}$	$k_{1,2}$	$k_{1,3}$
$k_{2,0}$	$k_{2,1}$	$k_{2,2}$	$k_{2,3}$
$k_{3,0}$	$k_{3,1}$	$k_{3,2}$	$k_{3,3}$

It is very important to know that the cipher input bytes are mapped onto the the state bytes in the order $a_{0,0}$, $a_{1,0}$, a2,0, a3,0, a0,1, a1,1, a2,1, a3,1 ... and the bytes of the cipher key are mapped onto the array in the order k0,0, k1,0, k2,0, k3,0, k0,1, k1,1, k2,1, k3,1 ... At the end of the cipher operation, the cipher output is extracted from the state by taking the state bytes in the same order. AES uses a variable number of rounds, which are fixed: A key of size 128 has 10 rounds.During each round, the following operations are applied on the state:

- **SubBytes:** every byte in the state is replaced by another one, using the Rijndael S-Box.
- **ShiftRow:** every row in the 4x4 array is shifted a certain amount to the left.
- **MixColumn:** a linear transformation on the columns of the state.
- **AddRoundKey:** each byte of the state is combined with a round key, which is a different key for each round and derived from the Rijndael key schedule.

  In the final round, the MixColumn operation is omitted.

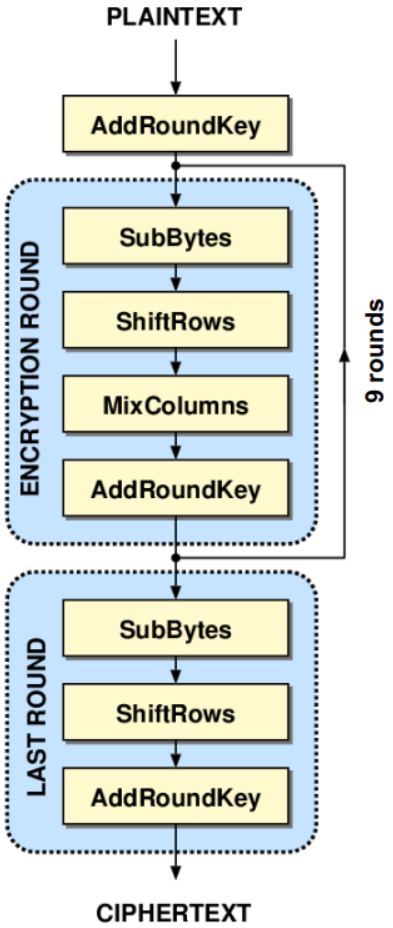


Figure 2: Caption

Observations:

- The cipher key is expanded into a larger key, which is later used for the actual operations.
- The roundKey is added to the state before starting the with loop.
- The FinalRound() is the same as Round(), apart from missing the MixColumns() operation.
- During each round, another part of the ExpandedKey is used for the operations.
- The ExpandedKey shall ALWAYS be derived from the Cipher Key and never be specified directly.

5 AES operations: SubBytes, ShiftRow, MixColumn and AddRoundKey

5.1 The AddRoundKey operation:

In this operation, a Round Key is applied to the state by a simple bitwise XOR. The Round Key is derived from the Cipher Key by the means of the key schedule. The Round Key length is equal to the block key length (=16 bytes).

$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} \oplus \begin{pmatrix} k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \end{pmatrix} = \begin{pmatrix} b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \end{pmatrix}$$

$$b_{i,j} = a_{i,j} \oplus k_{i,j}$$

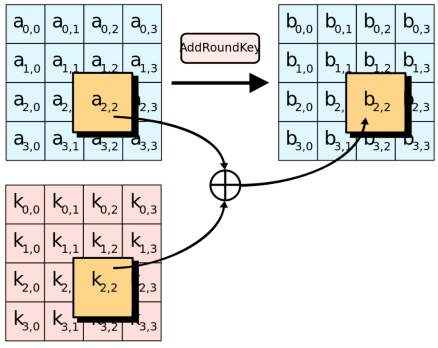


Figure 3: Add Round key

5.2 The ShiftRow operation:

In this operation, each row of the state is cyclically shifted to the left, depending on the row index.

- The 1st row is shifted 0 positions to the left.
- The 2nd row is shifted 1 positions to the left
- The 3rd row is shifted 2 positions to the left.
- The 4th row is shifted 3 positions to the left.

$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} \rightarrow \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} \\ a_{2,2} & a_{2,3} & a_{2,0} & a_{2,1} \\ a_{3,3} & a_{3,0} & a_{3,1} & a_{3,2} \end{pmatrix}$$

A visual representation of this operation can be found below:

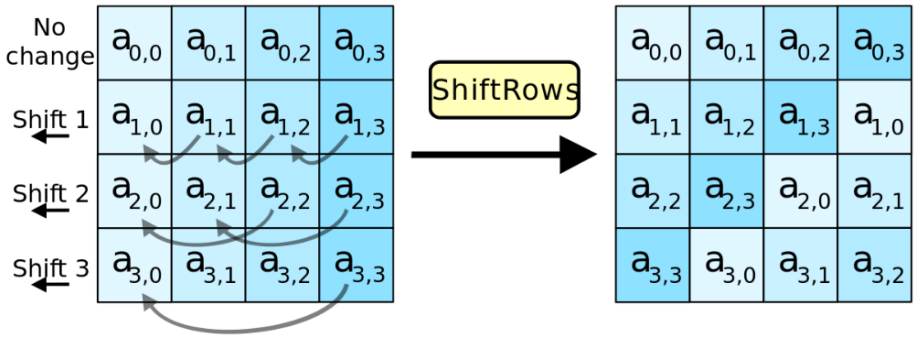


Figure 4: ShiftRow operation

5.3 The SubBytes operation:

The SubBytes operation is a non-linear byte substitution, operating on each byte of the state independently. The substitution table (S-Box) is invertible and is constructed by the composition of two transformations:

- Take the multiplicative inverse in Rijndael's finite field
- Apply an affine transformation which is documented in the Rijndael documentation.

Since the S-Box is independent of any input, pre-calculated forms are used, if enough memory (256 bytes for one S-Box) is available. Each byte of the state is then substituted by the value in the S-Box whose index corresponds to the value in the state:

$$a_{i,j} = \text{SBox}[a_{i,j}]$$

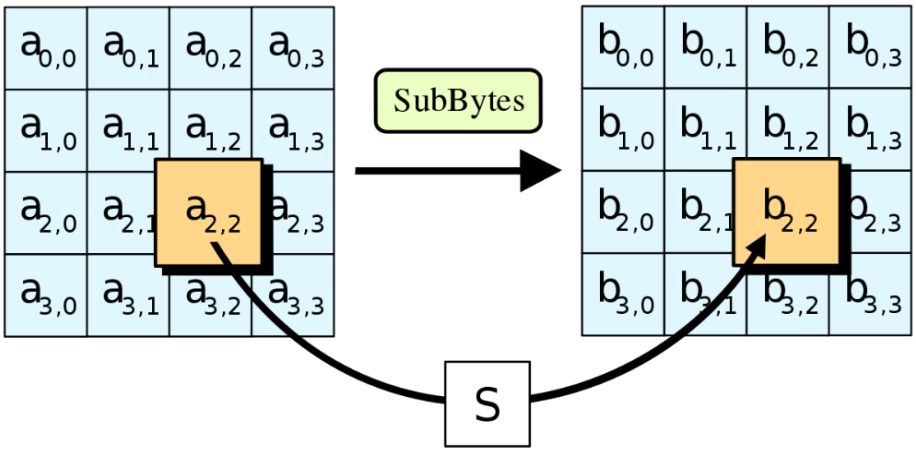


Figure 5: SubBytes

The inverse of SubBytes is the same operation, using the inversed S-Box, which is also precalculated.

5.4 The MixColumn operation:

Addition and Subtraction: Addition and subtraction are performed by the exclusive or operation. The two operations are the same; there is no difference between addition and subtraction.
Multiplication in Rijndael's galois field is a little more complicated. The procedure is as follows:

- Take two eight-bit numbers, a and b, and an eight-bit product p
- Set the product to zero
- Make a copy of a and b, which we will simply call a and b in the rest of this algorithm

Run the following loop eight times:

1. If the low bit of b is set, exclusive or the product p by the value of a
2. Keep track of whether the high (eighth from left) bit of a is set to one

3. Rotate a one bit to the left, discarding the high bit, and making the low bit have a value of zero
4. If a's hi bit had a value of one prior to this rotation, exclusive or a with the hexadecimal number 0x1b
5. Rotate b one bit to the right, discarding the low bit, and making the high (eighth from left) bit have a value of zero

The product p now has the product of a and b.

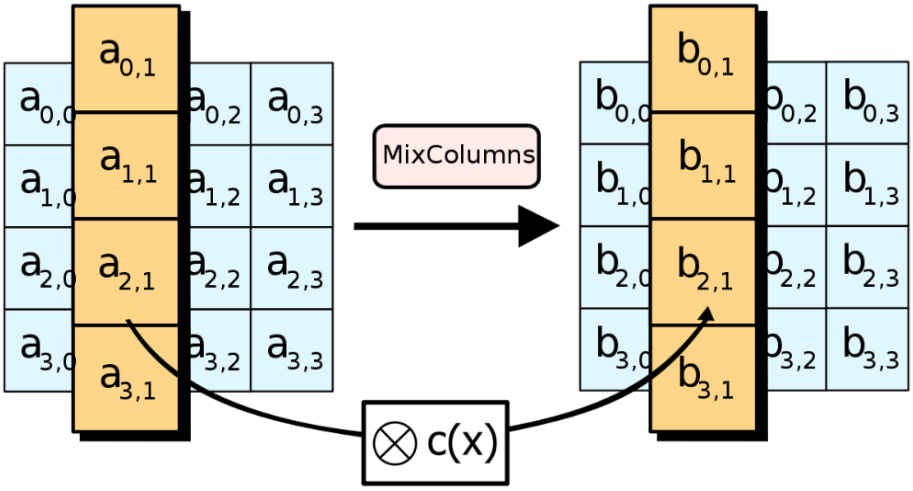


Figure 6: Mixcolumns

Example:

$$\begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix} \cdot \begin{bmatrix} 02 & 01 & 01 & 03 \\ 03 & 02 & 01 & 01 \\ 01 & 03 & 02 & 01 \\ 01 & 01 & 03 & 02 \end{bmatrix} = \begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix}$$

Here, the first element is calculated as:

$$(d4 \cdot 02) + (bf \cdot 03) + (5d \cdot 01) + (30 \cdot 01) = 04$$

First, we will try to solve $d4 \cdot 02$.
We will convert $d4$ to its binary form, where:

$$d4_{16} = 11010100_2$$

Now calculate:

$$d4 \cdot 02 = 11010100 \ll 1 = 10101000 \oplus 00011011 = 10110011$$

Where: - $\ll$ is left shift (1 is the number of bits to shift), - $\oplus$ is XOR (since the leftmost bit is 1 before the shift).

Detailed Calculation:

$$10101000 \oplus 00011011 = 10110011$$

Thus, $d4 \cdot 02 = 10110011$ (answer).

The binary value of $d4$ will be XORed with 00011011 after shifting, because the leftmost bit of $d4$ is equal to 1 before the shift.

6 The Rijndael Key Schedule:

The Key Schedule is responsible for expanding a short key into a larger keys. An 128 bit key is expanded to an 176 byte key.

There is a relation between the cipher key size, the number of rounds and the ExpandedKey size. For an 128-bit key, there is one initial AddRoundKey operation plus there are 10 rounds and each round needs a new 16 byte key, therefor we require 10+1 RoundKeys of 16 byte, which equals 176 byte. The same logic can be applied to the two other cipher key sizes. The general formula is that: ExpandedKeySize = (nbrRounds+1) * BlockSize

The Key Schedule is made up of iterations of the Key schedule core, which works on 4-byte words. The core uses a certain number of operations, which are explained here:

Rotate: The 4-byte word is cyclically shifted 1 byte to the left.

$1d$	$2c$	$3a$	$4f$	$\longrightarrow$	$2c$	$3a$	$4f$	$1d$

RCon: The Rcon values can be pre-calculated, which results in a simple substitution (a table lookup) in a fixed Rcon table.

SBox: The Key Schedule uses the same S-Box substitution as the main algorithm.

The Key Schedule Core:

Algorithm 1 keyScheduleCore

```
function KEYSCHEDULECORE(word)
    Rotate(word)
    SBoxSubstitution(word)
    word[0] = word[0] XOR RCON[i]
end function
```

In the above code, word has a size of 4 bytes and i is the iteration counter from the Key Schedule.

7 The Key Expansion:

```c
// Key expansion function
void key_expansion(unsigned char *key, unsigned char *expandedkey) {
    unsigned char tmp1[4]; //Contains the last word of the previous round key
    unsigned char tmp2[4]; //Contains the word from NK positions before in the expanded key
    uint8_t i, j;

    // Copy the original key into the beginning of expanded key
    //This copies the original 16-byte key (AES-128 key) into the first 16 bytes of the expandedkey array
    for (i = 0; i < NK; i++) {
        for (j = 0; j < 4; j++) {
            expandedkey[4 * i + j] = key[4 * i + j];
        }
    }

    // Key expansion loop
    //This loop generates words for the expanded key
    for (i = 4; i < 4 * (Nr + 1); i++) {
        for (j = 0; j < 4; j++) {
            tmp1[j] = expandedkey[4 * (i - 1) + j];
            tmp2[j] = expandedkey[4 * (i - NK) + j];
        }
        if (i % NK == 0) {
            rotate_word(tmp1); //Rotates the word, shifting the first byte to the end
            subword(tmp1);     //Applies the S-box substitution to each byte of tmp1
            tmp1[0] = tmp1[0] ^ Rcon[i / NK]; //Adds the round constant to the first byte of tmp1
        }
        for (j = 0; j < 4; j++) {
            expandedkey[4 * i + j] = tmp2[j] ^ tmp1[j];  //Generate the next word by XORing
        }
    }
}
```

Activate Window
Go to Settings to activ

Figure 7: Key Expansion

7.1 Parameter:

- unsigned char *key: The input original key (128 bits or 16 bytes for AES-128).
- unsigned char *expandedkey: The buffer where the expanded key will be stored.

7.2 Copy the original key to the expanded key:

- copies the original 16-byte key (AES-128 key) into the first 16 bytes of the expandedkey array.
- NK is the key size divided by 32 (in the case of AES-128, NK = 4)

7.3 Key Expansion Loop:

The rest of the expanded key schedule is computed by deriving new words from the previous words in the expanded key.

- This loop generates words for the expanded key.

- It starts at i = 4 because the first 16 bytes (4 words) are already set with the original key.
- tmp1: Contains the last word of the previous round key.
- tmp2: Contains the word from NK positions before in the expanded key.

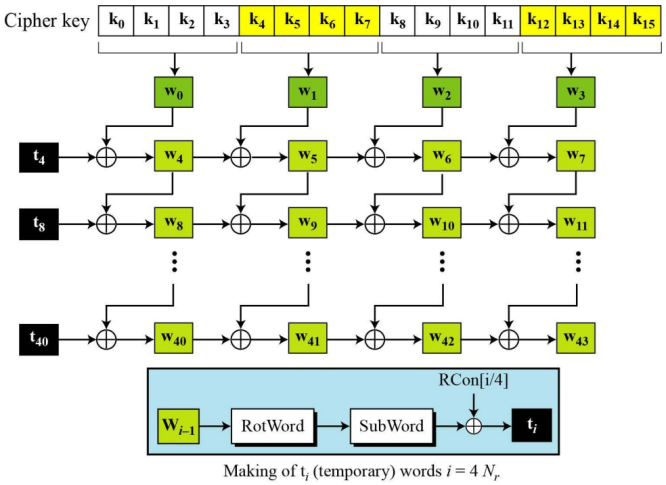
7.4 Perform Special Operations Every NK Iterations:

Every NK words (4 words for AES-128), a special transformation is applied.

- rotate word(tmp1): Rotates the word, shifting the first byte to the end.
- subword(tmp1): Applies the S-box substitution to each byte of tmp1.
- Rcon[i / NK]: Adds the round constant to the first byte of tmp1.

7.5 Generate the Next Word:

- The next word in the expanded key is generated by XORing tmp1 (the transformed word) and tmp2 (the word NK positions back in the expanded key).
- This process is repeated to generate all the required words for the expanded key.



8 Decryption:

For implementations on 8-bit platforms, there is no benefit in following the equivalent decryption algorithm. Instead, the straightforward decryption algorithm is followed. Decryption is similar in structure to encryption, but uses the InvMixColumns step instead of MixColumns. Where the MixColumns coefficients are limited to 01, 02, and 03, the coefficients of InvMixColumns are 09, $0E$, $0B$, and $0D$. In our implementation, these multiplications take significantly more time, resulting in a small performance degradation. A considerable speed-up can be obtained by using look-up tables at the cost of additional tables.

P. Barreto observed the following relation between the MixColumns polynomial $c(x)$ and the InvMixColumns polynomial $d(x)$:

$$d(x) = 04x^2 + 05 \cdot c(x) \mod (x^4 + 01)$$

In matrix notation, this relation becomes:

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} 05 & 00 & 04 & 00 \\ 00 & 05 & 00 & 04 \\ 04 & 00 & 05 & 00 \\ 00 & 04 & 00 & 05 \end{bmatrix}$$

The consequence is that InvMixColumns can be implemented as a simple preprocessing step, followed by a MixColumns step. An algorithm for the preprocessing step is given below. If the small performance drop caused by this implementation of the preprocessing step is acceptable, no extra tables have to be defined.
Let a be a column vector. The expression can be written as:

$$u = \text{xtime}\,(\text{xtime}\,(a[0] \oplus a[2]))$$
$$v = \text{xtime}\,(\text{xtime}\,(a[1] \oplus a[3]))$$
$$a[0] = a[0] \oplus u$$
$$a[1] = a[1] \oplus v$$
$$a[2] = a[2] \oplus u$$
$$a[3] = a[3] \oplus v$$

9 Modes Of Operation:

Among all the modes, the input to the encryption processes of the CBC, CFB, and OFB modes includes, in addition to the plaintext, a data block called the initialization vector, denoted as IV. The IV is used in an initial step in the encryption of a message and in the corresponding decryption of the message. Here we have used `openssl rand` to generate a random IV. For the ECB and CBC modes, the plaintext must be a sequence of one or more complete data blocks. In order to make the plaintext length a multiple of the number of bytes in a block, we have used `PKCS#7` padding padding.

1. Electronic Codebook Mode (ECB mode)
2. Cipher Blockchaining Mode (CBC mode)
3. Output Feedback Mode (OFB mode)
4. Cipher Feedback Mode (CFB mode)
5. Counter Mode (CTR mode)

Out of five modes we will discuss only first four modes.

9.1 ECB mode:

ECB mode represents the straightforward use of a block cipher. Given a sequence of plaintext blocks $m_0, m_1, \ldots$, each block m_i is encrypted using the same key K, producing a corresponding sequence of ciphertext blocks $c_0, c_1, \ldots$ (after applying `PKCS#7` padding). In practice, ECB mode is almost never used due to its significant weakness: identical plaintext blocks will always result in identical ciphertext blocks. This vulnerability is especially problematic when the plaintext consists of blocks from a low-entropy space. For example, if a block is made up entirely of 0's or 1's, then ECB mode offers little to no security.

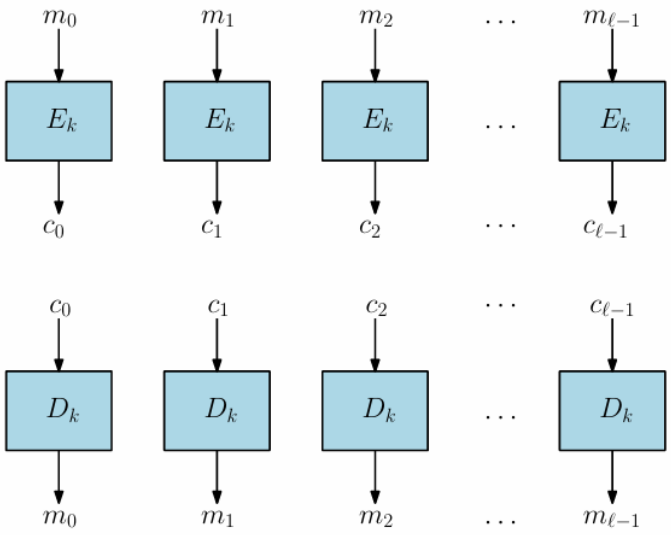


Figure 8: ECB mode

Encryption Algorithm:

Algorithm 2 Encryption(m, k)

1: **for** $i = 0$ to $\ell - 1$ **do**
2: $\quad c_i \leftarrow E_k(m_i) = E(k, m_i)$
3: **end for**
4: **return** $c = (c_0, c_1, \ldots, c_{\ell-1})$

Decryption Algorithm:

Algorithm 3 Decryption(c, k)

1: **for** $i = 0$ to $\ell - 1$ **do**
2: $\quad m_i \leftarrow E_k^{-1}(m_i) = D(k, c_i)$
3: **end for**
4: **return** $m = (m_0, m_1, \ldots, m_{\ell-1})$

9.2 CBC mode:

In Cipher Block Chaining (CBC) mode, each ciphertext block c_i is XOR-ed with the next plaintext block m_{i+1} before being encrypted with the key K. More formally, we begin with an initialization vector (IV), denoted by IV. Note that the IV has the same length as a plaintext block.

The ciphertext blocks $c_0, c_1, \ldots$ are then constructed using the following rule:

$$c_0 = E_K(m_0 \oplus \text{IV})$$

$$c_i = E_K(m_i \oplus c_{i-1}) \quad \text{for } i \geq 1$$

$E_K(x)$ is the encryption of block x using the key K. Each block of plaintext is XOR-ed with the previous ciphertext block (starting with the IV for the first block), and the result is encrypted with the key K.

Encryption Algorithm:

Algorithm 4 Encryption(m, k)

1: Choose a random IV
2: Compute $c_0 \leftarrow E_k(\text{IV} \oplus m_0) = E(k, \text{IV} \oplus m_0)$
3: **for** $i = 1$ to $\ell - 1$ **do**
4: $\quad$ Compute $c_i \leftarrow E_k(m_i \oplus c_{i-1}) = E(k, m_i \oplus c_{i-1})$
5: **end for**
6: **return** (IV, c), where $c = (c_0, c_1, \ldots, c_{\ell-1})$

Decryption Algorithm:

Algorithm 5 Decryption((IV, c), k)

1: Compute $m_0 \leftarrow D_k(c_0) \oplus \text{IV} = D(k, c_0) \oplus \text{IV}$
2: **for** $i = 1$ to $\ell - 1$ **do**
3: Compute $m_i \leftarrow D_k(c_i) \oplus c_{i-1} = D(k, c_i) \oplus c_{i-1}$
4: **end for**
5: **return** $m = (m_0, m_1, \ldots, m_{\ell-1})$

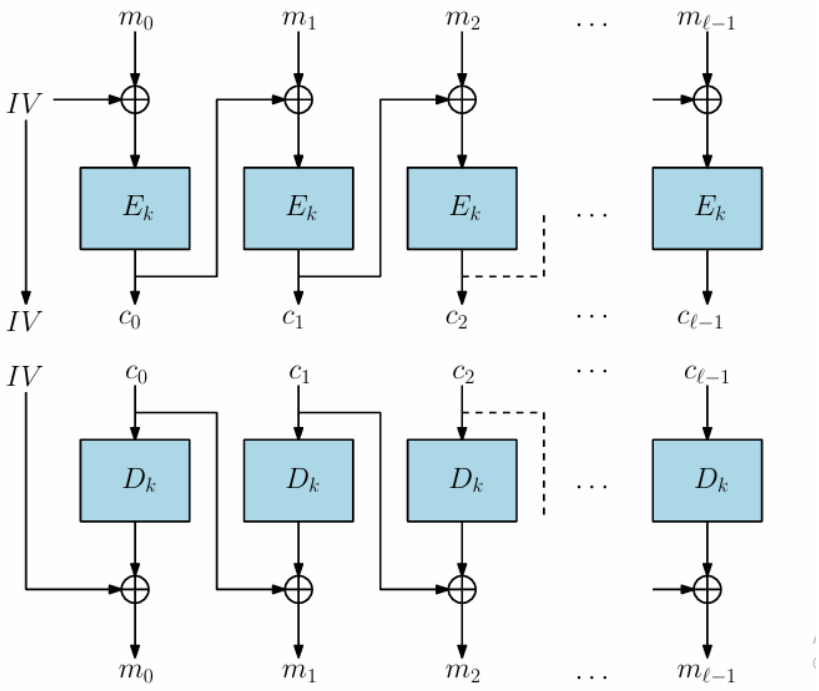


Figure 9: CBC mode

9.3 CFB mode:

ciphertext feedback uses an initialization vector (IV). CFB uses a block cipher as a component of a random number generator. In CFB mode, the previous ciphertext block is encrypted and the output is XORed with the current plaintext block to create the current ciphertext block. The XOR operation conceals plaintext patterns. Plaintext cannot be directly worked on unless the blocks from either the beginning or end of the ciphertext are retrieved

Encryption

Algorithm 6 Encryption(m, k)

1: Choose a random $IV \in \mathcal{R} \in \chi$
2: $c_0 := E_k(IV) \oplus m_0 = E(k, IV) \oplus m_0$
3: **for** $i = 1, \ldots, \ell - 1$ **do**
4: Compute $c_i := E_k(c_{i-1}) \oplus m_i = E(k, c_{i-1}) \oplus m_i$
5: **end for**
6: Return (IV, c), where $c = (c_0, c_1, \ldots, c_{\ell-1})$

Decryption

Algorithm 7 Decryption(IV, c)

1: $m_0 := E_k(IV) \oplus c_0 = E(k, IV) \oplus c_0$
2: **for** $i = 1, \ldots, \ell - 1$ **do**
3: Compute $m_i := E_k(c_{i-1}) \oplus c_i = E(k, c_{i-1}) \oplus c_i$
4: **end for**
5: Return $m = (m_0, m_1, \ldots, m_{\ell-1})$

9.4 OFB mode:

OFB (**Output Feedback**) is an AES block cipher mode similar to the CFB mode. What mainly differs from CFB is that the OFB mode relies on XORing plaintext and ciphertext blocks with expanded versions of the initialization vector.

This process can be seen as a one-time pad and the expanded vectors as *pad vectors*. The following formula depicts how a sequence of pad vectors is created:

$$V_i = E_K(V_{i-1}) \tag{1}$$

where E_K denotes the block encryption algorithm using key K, and V_i and V_{i-1} are adjacent vectors.

In the formula above, we are assuming V_0 to be the initialization vector.

Once the sequence of pad vectors is generated, encryption with the OFB mode can be carried out using the following formula:

$$C_i = V_i \oplus B_i \tag{2}$$

Decryption is carried out in a similar way:

$$B_i = V_i \oplus C_i \tag{3}$$

Like the CFB mode, OFB also makes use of a single encryption algorithm for both encryption and decryption.

Encryption

Algorithm 8 Encryption(m, k)

1: Choose a random $IV \in \mathcal{R} \in \chi$
2: $y_0 := E_k(IV) = E(k, IV)$; $c_0 := y_0 \oplus m_0$
3: **for** $i = 1, \ldots, \ell - 1$ **do**
4: Compute $y_i := E_k(y_{i-1}) = E(k, y_{i-1})$
5: Compute $c_i := y_i \oplus m_i$
6: **end for**
7: Return (IV, c), where $c = (c_0, c_1, \ldots, c_{\ell-1})$

Decryption

Algorithm 9 Decryption(IV, c)

1: $y_0 := E_k(IV) = E(k, IV)$; $m_0 := y_0 \oplus c_0$
2: **for** $i = 1, \ldots, \ell - 1$ **do**
3: Compute $y_i := E_k(y_{i-1}) = E(k, y_{i-1})$
4: Compute $m_i := y_i \oplus c_i$
5: **end for**
6: Return $m = (m_0, m_1, \ldots, m_{\ell-1})$